$i \rightarrow$ index of S $\quad m = S.size()$

$j \rightarrow$ index of T $\quad n = T.size()$

DP[m][n]

```
int f( i, j, DP){
    Base case:
     if(i<0)  return  j*I
     if(j<0)  return  i*D

     if (DP[i][j] != -1) return DP[i][j]

     if(S[i] == T[j])
         DP[i][j] = f(i-1, j-1, DP)
         return f(i-1, j-1, DP)

     i_cost = I + f(i, j-1)
     r_cost = R + f(i-1, j-1)
     d_cost = D + f(i-1, j)

     DP[i][j] = min(i_cost, r_cost, d_cost)
     return DP[i][j]
}
```

$$\therefore \text{Recurrence } DP[i][j] = \begin{cases} DP[i-1][j-1]; & \text{if } S[i] == T[j] \\ \min(DP[i, j-1], \\ \quad DP[i-1][j-1], \\ \quad DP[i-1][j]) \end{cases}$$

Base Case:

$DP[0][j] = j \times I$

$DP[i][0] = i \times D$

Time Complexity:

DP: nested for loops of size m & n.

So, the time complexity is $O(m \times n)$

Backtracking:

A while loop until $(i<0 \,||\, j<0)$

$i + j = m + n$ at maximum.

So the loop can run at most $m+n$ times.

$\therefore O(m+n)$

$\therefore$ Overall time complexity $O(m \times n)$